

```

1  int HorseRunningAction = 1; // 跑马灯动作模式：模式1，模式2
2  int KeyDLTime = 0;          // 按键去抖计时计数器
3  // 外部中断1初始化，配置GPIO12作为外部中断源，上升沿和下降沿都触发
4  void Xint1_Init()
5  {
6      EALLOW;                  // 关闭写保护，允许修改受保护寄存器
7      SysCtrlRegs.PCLKCR3.bit.GPIOINENCLK = 1; // 使能GPIO输入时钟
8      GpioCtrlRegs.GPAMUX1.bit.GPIO12 = 0;    // 配置GPIO12为普通数字IO功能
9      GpioCtrlRegs.GPADIR.bit.GPIO12 = 0;      // 输入方向
10     GpioCtrlRegs.GPAPUD.bit.GPIO12 = 1;       // 使/禁能上拉电阻，看板子情况
11     GpioCtrlRegs.GPAQSEL1.bit.GPIO12 = 0;     // 采样与系统时钟同步
12     GpioDataRegs.GPACLEAR.bit.GPIO12 = 1;     // 清除GPIO12数据位
13
14     GpioIntRegs.GPIOXINT1SEL.bit.GPIOSEL = 12; // 中断源选择
15     XIntruptRegs.XINT1CR.bit.POLARITY = 3;     // 双边沿触发（上升沿和下降沿都触发）
16     XIntruptRegs.XINT1CR.bit.ENABLE = 1;       // 使能XINT1
17     EDIS;                                       // 使能写保护
18 }
19 // 跑马灯LED的GPIO初始化
20 void HorseIO_Init()
21 {
22     EALLOW;
23     GpioDataRegs.GPASET.bit.GPIO0 = 1; // 输出高电平
24     GpioDataRegs.GPASET.bit.GPIO1 = 1;
25     GpioDataRegs.GPASET.bit.GPIO2 = 1;
26     GpioDataRegs.GPASET.bit.GPIO3 = 1;
27
28     GpioCtrlRegs.GPAMUX1.bit.GPIO0 = 0;
29     GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 0;
30     GpioCtrlRegs.GPAMUX1.bit.GPIO2 = 0;
31     GpioCtrlRegs.GPAMUX1.bit.GPIO3 = 0;
32
33     GpioCtrlRegs.GPADIR.bit.GPIO0 = 1; // 设置GPIO0-GPIO3为输出方向
34     GpioCtrlRegs.GPADIR.bit.GPIO1 = 1;
35     GpioCtrlRegs.GPADIR.bit.GPIO2 = 1;
36     GpioCtrlRegs.GPADIR.bit.GPIO3 = 1;
37     EDIS;
38 }
39 void main(void)
40 {
41     InitSysCtrl(); // 初始化系统控制：PLL,看门狗、等
42     InitGpio();
43     DINT;          // 关闭总中断
44     IER = 0x0000;  // 关闭CPU中断使能
45     IFR = 0x0000;  // 清除CPU中断标志
46     InitPieCtrl(); // 初始化PIE中断
47     InitPieVectTable(); // 初始化PIE中断向量表
48     EALLOW;
49     // 配置中断向量表
50     PieVectTable.TINT0 = &cpu_timer0_isr;
51     PieVectTable.TINT1 = &cpu_timer1_isr;
52     PieVectTable.XINT1 = &myXint1_isr;

```

```

53     EDIS;
54
55     InitCpuTimers();
56     ConfigCpuTimer(&CpuTimer0, 60, 500000); // 周期500ms; 5*10^(5-6), 用于跑
马灯
57     ConfigCpuTimer(&CpuTimer1, 60, 10000); // 周期10ms, 用于按键去抖计时
58
59     EALLOW;
60     CpuTimer0Regs.TCR.all = 0x4001; // 定时器0中断使能, TCR.TIE=1
61     CpuTimer1Regs.TCR.all = 0x4001; //
62     EDIS;
63     HorseIO_Init();
64     Xint1_Init();
65     TM1638_Init(); // 初始化TM1638数码管驱动芯片
66     // 使能CPU中断
67     IER |= M_INT1;
68     IER |= M_INT13; // timer1
69     // 使能PIE组1中的中断
70     PieCtrlRegs.PIEIER1.bit.INTx7 = 1; // TINT0
71     PieCtrlRegs.PIEIER1.bit.INTx4 = 1; // XINT1
72     EINT; // 使能全局中断
73     ERTM; // 实时调试模式
74     while (1)
75     {}
76 }
77
78 void HorseRunning1(int no)
79 {
80     if (no & 0x1)
81         Led0Blink(); // bit0为1则点亮LED0
82     else
83         Led0Blank();
84     if (no & 0x2)
85         Led1Blink(); // bit1为1则点亮LED1
86     else
87         Led1Blank();
88     if (no & 0x4)
89         Led2Blink(); // bit2为1则点亮LED2
90     else
91         Led2Blank();
92     if (no & 0x8)
93         Led3Blink(); // bit3为1则点亮LED3
94     else
95         Led3Blank();
96 }
97
98 void HorseRunning2(int no)
99 {
100     no = no % 0x5;
101     if (no == 0x1)
102         Led0Blink();
103     else
104         Led0Blank();
105     if (no == 0x2)
106         Led1Blink();
107     else

```

```

108         Led1Blank();
109     if (no == 0x3)
110         Led2Blink();
111     else
112         Led2Blank();
113     if (no == 0x4)
114         Led3Blink();
115     else
116         Led3Blank();
117 }
118
119 // 外部中断1服务函数（按键GPIO12）跑马灯模式切换
120 interrupt void myXint1_isr(void)
121 {
122     if (GpioDataRegs.GPADAT.bit.GPIO12 == 1)
123     {
124         CpuTimer1Regs.TCR.all = 0x4001; // 按键释放则无事发生
125     }
126     else
127     { // 按键按下状态（GPIO12为低电平）
128         // 如果按下稳定超过200ms，则认为是有效按键
129         if (KeyDLTime > 20)
130         {
131             CpuTimer0.InterruptCount = 0; // 复位定时器0计数，重
132             新开始跑马灯
133             HorseRunningAction = !HorseRunningAction; // 切换跑马灯动作模式
134         }
135         CpuTimer1Regs.TCR.bit.TRB = 1; // 重载PRD
136         CpuTimer1Regs.TCR.bit.TSS = 0; // 启动定时器
137         KeyDLTime = 0;
138     }
139     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1; // 清除PIE组1的中断应答标志，允许
140     同组其他中断响应
141 }
142
143 // 定时器0中断服务函数，周期：500ms，更新LED显示状态
144 interrupt void cpu_timer0_isr(void)
145 {
146     CpuTimer0.InterruptCount++; // 中断计数自增，用于跑马灯移位
147     if (HorseRunningAction == 0)
148     {
149         HorseRunning1(CpuTimer0.InterruptCount); // 模式1：逐位累加点亮
150     }
151     else
152     {
153         HorseRunning2(CpuTimer0.InterruptCount); // 模式2：单灯循环移动
154     }
155     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
156 }
157
158 // 定时器1中断服务函数，周期：10ms。按键去抖计时，每10ms计数器加1
159 interrupt void cpu_timer1_isr(void)
160 {
161     KeyDLTime++;
162 }

```

```

1 void ECap_Init(void)
2 {
3     ECap1Regs.ECEINT.all = 0x0000;    // 禁捕获中断
4     ECap1Regs.ECCLR.all = 0xFFFF;    // 清除标志
5     ECap1Regs.ECCTL1.bit.CAPLDEN = 0; // 禁止CAP1-CAP4
6     ECap1Regs.ECCTL2.bit.TSCTRSTOP = 0; // 停止计数器
7
8     // 配置捕获模式和工作方式
9     ECap1Regs.ECCTL2.bit.CONT_ONESHT = 1; // 单次触发模式
10    ECap1Regs.ECCTL2.bit.STOP_WRAP = 3;    // 4个事件后停止
11    //
12    ECap1Regs.ECCTL1.bit.CAP1POL = 1; // 下降沿捕获
13    ECap1Regs.ECCTL1.bit.CAP2POL = 0; // 上升沿捕获
14    ECap1Regs.ECCTL1.bit.CAP3POL = 1;
15    ECap1Regs.ECCTL1.bit.CAP4POL = 0;
16    ECap1Regs.ECCTL1.bit.CTRRST1 = 0; // 绝对模式
17    ECap1Regs.ECCTL1.bit.CTRRST2 = 0;
18    ECap1Regs.ECCTL1.bit.CTRRST3 = 0;
19    ECap1Regs.ECCTL1.bit.CTRRST4 = 0;
20
21    ECap1Regs.ECCTL2.bit.SYNCI_EN = 1; // 使能同步输入
22    ECap1Regs.ECCTL2.bit.SYNCO_SEL = 0; // 同步输出
23    ECap1Regs.ECCTL1.bit.CAPLDEN = 1;    // 使能CAP
24    ECap1Regs.ECCTL2.bit.TSCTRSTOP = 1; // 启动计数器
25    ECap1Regs.ECCTL2.bit.REARM = 1;      // 重新产生单触发
26    ECap1Regs.ECCTL1.bit.CAPLDEN = 1;
27
28    ECap1Regs.ECEINT.bit.CEVT4 = 1; // 使能第4个事件中断
29 }
30 // EPWM1初始化函数，输出带死区的互补PWM波形
31 void EPwm1_Init(void)
32 {
33     EPwm1Regs.TBPRD = 4000;            // 周期 3750KHZ/3000/2
34     EPwm1Regs.TBPHS.half.TBPHS = 0x0000; // 相位为0
35     EPwm1Regs.TBCTR = 0x0000;        // 清除计数器
36     // 时基控制寄存器配置
37     EPwm1Regs.TBCTL.bit.CTRMODE = 2;    // 先增后减
38     EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // 禁相位加载
39     EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV4; // 高速外设时钟分频：4分频
40     EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV4; // 时钟分频：4分频，此时TBCLK是
41     60M/16= 3750KHZ
42
43     EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW; // 启用影子寄存器
44     EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
45     EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // 计数器为零时装载比较值
46     EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;
47     // 占空比配置
48     EPwm1Regs.CMPA.half.CMPA = 1000; // 比较值 = 周期的一半，产生50%占空比
49     EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
50     EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
51     // 死区配置
52     EPwm1Regs.DBCTL.bit.OUT_MODE = 3; // 完全使能
53     EPwm1Regs.DBCTL.bit.IN_MODE = 0;
54     EPwm1Regs.DBCTL.bit.POLSEL = 2; // 互补高电平有效
55     // 死区长度

```

```

55     EPwm1Regs.DBRED = EPWM1_MIN_DB;
56     EPwm1Regs.DBFED = EPWM1_MIN_DB;
57 }
58 void main(void)
59 {
60     InitsSysCtrl();
61     DINT;
62     IER = 0x0000;
63     IFR = 0x0000;
64     InitPieCtrl();
65     InitPieVectTable();
66     // 配置中断向量表
67     EALLOW;
68     PieVectTable.ECAP1_INT = &Ecap1Int_isr;
69     EDIS;
70     InitFlash();
71     InitCpuTimers();
72     ConfigCpuTimer(&CpuTimer0, 60, 500000);
73     ConfigCpuTimer(&CpuTimer1, 60, 10000);
74     EALLOW;
75     SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 0; // 禁能ePWM模块的时钟
76     EDIS;
77     TM1638_Init();
78     ECap_Init();
79     EPwm1_Init();
80     InitLEDGpio();
81     InitEPwm1Gpio(); // 初始化EPWM、ECAP的GPIO引脚输入输出，见多路复用MUX图，
                        // GPIO0-EPWM1A, GPIO19给ECAP
82     InitECap1Gpio();
83     EALLOW;
84     SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1; // 使能ePWM模块时钟同步
85     EDIS;
86     // 使能中断
87     IER |= M_INT4; // eCAP中断
88     // IER |= M_INT3; // EPWM中断
89     PieCtrlRegs.PIEIER4.bit.INTx1 = 1; // 使能PIE组4的第1个中断（eCAP1）
90     PieCtrlRegs.PIEIER3.bit.INTx1 = 1; // 使能PIE组3的第1个中断（EPWM1）
91     EINT;
92     ERTM;
93 }
94
95 // eCAP1捕获中断服务函数，完成4个捕获事件后（下降沿、上升沿、下降沿、上升沿）
96 // ，计算捕获的PWM信号周期和占空比，并在数码管上显示
97 interrupt void Ecap1Int_isr(void)
98 {
99     float Tst1 = 0; // CAP1捕获的时间戳（下降沿）
100    float Tst2 = 0; // CAP2捕获的时间戳（上升沿）
101    float Tst3 = 0; // CAP3捕获的时间戳（下降沿）
102    float Tst4 = 0; // CAP4捕获的时间戳（上升沿）
103    float Period1 = 0; // 计算得到的PWM周期
104    float Duty = 0; // 计算得到的占空比
105    // 读取4个捕获寄存器中的时间戳值
106    Tst1 = ECap1Regs.CAP1;
107    Tst2 = ECap1Regs.CAP2;
108    Tst3 = ECap1Regs.CAP3;
109    Tst4 = ECap1Regs.CAP4;

```

```

110 // 计算周期：两次下降沿之间的时间差
111 Period1 = Tst3 - Tst1; // 周期 = 下降沿3 - 下降沿1 理论值为3000*2=6000;
112 // 计算占空比：高电平时间 / 周期
113 // 高电平时间 = 上升沿2 - 下降沿1
114 Duty = (Tst2 - Tst1) / Period1; // 占空比 = 高电平时间 / 周期
115 ECap1Regs.ECCLR.bit.CEVT4 = 1; // 清除第4个事件标志
116 ECap1Regs.ECCLR.bit.INT = 1; // 清除中断标志
117 ECap1Regs.ECCTL2.bit.REARM = 1; // 重新准备下次捕获序列
118 LED_showfloat(5, Duty); // 显示占空比
119 PieCtrlRegs.PIEACK.all = PIEACK_GROUP4;
120 }

```

```

1 static float a[2]; // 一阶低通滤波数组：a[0]=当前值，a[1]=上一次值
2 interrupt void adc1_isr(void)
3 {
4     float v;
5     Voltage = AdcResult.ADCRESULT0; // 获取ADC通道0的转换结果，12位 4096
6     // 原理，从网上搜索公式为， $Y_n = a * X_n + (1-a) * Y_{n-1}$ ，直接取a=0.1
7     a[1] = a[0]; // 将当前值保存为历史值
8     a[0] = (float)Voltage; // 更新当前采样值
9     // 数字量换算为模拟电压值
10    v = (a[1] * 0.9f + a[0] * 0.1f) / 4096.f * 3.3f; // 因为参考电压3.3
11    LED_showfloat(6, v);
12    AdcRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; // 清除中断标志
13    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1; //
14    return;
15 }
16 // EPWM1初始化函，和实验2中一样的。
17 // ADC的转换触发采用PWM控制
18 void InitEPwm1()
19 {
20     EPwm1Regs.TBPRD = 6000;
21     EPwm1Regs.TBPHS.half.TBPHS = 0x0000;
22     EPwm1Regs.TBCTR = 0x0000; // 清除计数器
23     EPwm1Regs.TBCTL.bit.CTRMODE = 2; // 计数模式，产生对cheng
24     EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // 禁相位加载
25     EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV4; // 分频
26     EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV4;
27     EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
28     EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
29     EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
30     EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;
31     EPwm1Regs.CMPA.half.CMPA = 3000;
32     EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
33     EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
34 }
35
36 void main()
37 {
38     InitSysCtrl();
39     InitEPwm1Gpio();
40     TM1638_Init();
41     DINT;
42     IER = 0x0000;
43     IFR = 0x0000;

```

```

44     InitPieCtrl();
45     InitPieVectTable();
46     EALLOW;
47     PieVectTable.ADCINT1 = &adc1_isr;
48     EDIS;
49     InitAdc();
50     EALLOW;
51     SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 0; // 关ePWM模块时钟
52     EDIS;
53     InitEPwm1();
54     EALLOW;
55     // 6.1 ADC中断触发配置
56     AdcRegs.ADCCTL1.bit.INTPULSEPOS = 1;
57     AdcRegs.INTSEL1N2.bit.INT1E = 1; // 使能ADCINT1中断
58     AdcRegs.INTSEL1N2.bit.INT1CONT = 0; // 关闭ADCINT1连续模式
59     AdcRegs.INTSEL1N2.bit.INT1SEL = 0; // 选择EOC0触发ADCINT1
60     AdcRegs.ADCSOC0CTL.bit.CHSEL = 7; // 选择ADC通道7
61     AdcRegs.ADCSOC0CTL.bit.TRIGSEL = 5; // 触发源选 EPWM1A (
62     AdcRegs.ADCSOC0CTL.bit.ACQPS = 6; // 采样7个周期
63     EDIS;
64     // 使能PIE中的ADC中断
65     PieCtrlRegs.PIEIER1.bit.INTx1 = 1; // 使能PIE组1的第1个中断 (ADCINT1)
66     IER |= M_INT1;
67     // PWM触发ADC配置
68     EPwm1Regs.ETSEL.bit.SOCAEN = 1; // 使能SOCA
69     EPwm1Regs.ETSEL.bit.SOCASEL = 1; // 计数值为0时触发。
70     // EPwm1Regs.ETSEL.bit.SOCASEL = 0x100;
71     EPwm1Regs.ETPS.bit.SOCAPRD = 1; // 每个匹配事件都触发一次
72     // EPwm1Regs.CMPA.half.CMPA = 0x0080; // 比较值128
73     EPwm1Regs.TBPRD = 0xFFFF; // 设置PWM周期
74     EPwm1Regs.TBCTL.bit.CTRMODE = 0; // 设定计数模式：向上计数
75     IER |= M_INT4;
76     PieCtrlRegs.PIEIER3.bit.INTx1 = 1;
77     PieCtrlRegs.PIEIER4.bit.INTx1 = 1;
78     EALLOW;
79     SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1; // 启动ePWM模块时钟
80     EDIS;
81     EINT;
82     ERTM;
83 }

```

```

1  float e_i = 0;
2  static float input[2];
3  static float output[2];
4  Uint16 inpu;
5  Uint16 outp;
6  // EPWM1 用于触发ADC采样，作为采样触发器
7  void InitEPwm1()
8  {
9      EPwm1Regs.TBPRD = 6000; //
10     EPwm1Regs.TBPHS.half.TBPHS = 0x0000; //相位
11     EPwm1Regs.TBCTR = 0x0000;
12     EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // 递增计数
13     EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; //禁相位
14     EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW; //

```

```

15     EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // 计数值为0加载影子reg
16     EPwm1Regs.CMPA.half.CMPA = 3000; // 比较值
17     EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR; // 递增计数到比较值输出低电平
18     EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET;
19     EPwm1Regs.ETSEL.bit.SOCAEN = 1; // 使能SOC信号
20     EPwm1Regs.ETSEL.bit.SOCASEL = 1; // 计数值为0时触发一次脉冲信号
21     EPwm1Regs.ETPS.bit.SOCAPRD = 1; // 每个匹配事件都触发一次SOC
22 }
23 // EPWM4同样的配置，用于控制输出电压，通过改变占空比调节输出电压大小
24 void InitEPwm4()
25 {
26     EPwm4Regs.TBPRD = 3000;
27     EPwm4Regs.TBPHS.half.TBPHS = 0x0000;
28     EPwm4Regs.TBCTR = 0x0000;
29     EPwm4Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
30     EPwm4Regs.TBCTL.bit.PHSEN = TB_DISABLE;
31     EPwm4Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
32     EPwm4Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
33     EPwm4Regs.CMPA.half.CMPA = 1500;
34     EPwm4Regs.AQCTLA.bit.CAU = AQ_CLEAR;
35     EPwm4Regs.AQCTLA.bit.ZRO = AQ_SET;
36     EPwm4Regs.ETSEL.bit.SOCAEN = 1;
37     EPwm4Regs.ETSEL.bit.SOCASEL = 1;
38     EPwm4Regs.ETPS.bit.SOCAPRD = 1;
39 }
40
41 void main()
42 {
43
44     InitsysCtrl();
45     InitEPwm1Gpio();
46     InitEPwm4Gpio();
47     TM1638_Init();
48     DINT;
49     InitPieCtrl();
50     IER = 0x0000;
51     IFR = 0x0000;
52     InitPieVectTable();
53     EALLOW;
54     PieVectTable.ADCINT1 = &Adc_isr;
55     EDIS;
56     InitAdc();
57     InitLEDGpio();
58     InitEPwm1();
59     InitEPwm4();
60     EALLOW;
61     SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1;
62     EDIS;
63     EALLOW;
64     //下面和实验3是一样的，就是配置EPWMA来触发ADC采样。
65     AdcRegs.ADCSAMPLEMODE.bit.SIMULEN0 = 1; // 使能SOC0的同步采样模式，
        SOC0\SOC1两路！
66     // ADC模块配置
67     AdcRegs.ADCCTL1.bit.INTPULSEPOS = 1;
68     AdcRegs.INTSEL1N2.bit.INT1E = 1; // 使能ADCINT1
69     AdcRegs.INTSEL1N2.bit.INT1CONT = 0;

```



```

70     AdcRegs.INTSEL1N2.bit.INT1SEL = 0; // 选EOC0触发ADCINT1
71     AdcRegs.ADCSOC0CTL.bit.CHSEL = 7; // 同步采样模式，选ADCINA7和ADCINB7，
    对应硬件图中的。前者采样如同实验3中的RP1电压，后者采样由epwm4的控制的电压。
72     AdcRegs.ADCSOC0CTL.bit.TRIGSEL = 5; // 触发源EPWM1A
73     AdcRegs.ADCSOC0CTL.bit.ACQPS = 6;
74     EDIS;
75     IER |= M_INT1; // ??CPU???1?ADC???
76     PieCtrlRegs.PIEIER1.bit.INTx1 = 1; // ??PIE?1??1????ADCINT1?
77     EINT;
78     ERTM;
79 }
80
81 interrupt void Adc_isr(void)
82 {
83
84     float in, out;
85     static float u_prev = 0; // 上一次输出值（增量式PI用）
86     // 5次滑动平均滤波变量
87     static float in_buf[5] = {0};
88     static int in_idx = 0;
89     static float in_sum = 0;
90     // 读取ADC转换结果
91     // 将ADC结果（0-4095）转换为实际电压值（0-3.3V）
92     inpu = AdcResult.ADCRESULT0; // 读取ADC转换结果（原始12位数值，范围0-4095）
93     outp = AdcResult.ADCRESULT1;
94     output[1] = output[0];
95     output[0] = (float)outp;
96     // 5次滑动平均滤波
97     in_sum = in_sum - in_buf[in_idx] + (float)inpu; // sum = 原本sum - 要丢掉的buf + 刚测的
98     in_buf[in_idx] = (float)inpu;
99     in_idx = (in_idx + 1) % 5; // in_idx + 1，从0 1 2 3 4循环
100    in = (in_sum / 5.0f) / 4096.f * 3.3f;
101    out = (output[1]*0.95f+output[0]*0.05f) / 4096.f * 3.3f;
102    // 增量式PI控制器
103    // 增量式PI公式:  $\Delta u = K_p(e - e_{last}) + K_i * e$ 
104    //  $u(k) = u(k-1) + \Delta u$ 
105    float delta_u; // 输出增量
106    float e; // 当前误差
107    static float e_last = 0; // 上一次误差（k-1时刻的误差）
108    // 计算误差 = 目标值 - 当前值
109    e = in - out;
110    //  $e(k-1) = e(k)$ 
111    e_last = e;
112    // 增量式PI计算
113    float Kp = 0.2f; // 比例系数（可调）
114    float Ki = 0.10f; // 积分系数（可调）
115    delta_u = Kp * (e - e_last) + Ki * e;
116    // 更新输出值  $u(k) = u(k-1) + \Delta u$ 
117    u_prev = u_prev + delta_u;
118    // 输出限幅（0-3.3V）
119    if (u_prev > 3.3f)
120    {
121        u_prev = 3.3f;
122    }
123    if (u_prev < 0)

```

```

124     {
125         u_prev = 0;
126     }
127     // 更新EPWM4占空比，占空比和比较值有关，把 0~3.3V 的电压 → 映射成 0~6000 的计数
    器值
128     // 当u=3.3V时，CMPA = TBPRD+1，占空比100%
129     // 当u=0V时，CMPA = 0，占空比0%
130     EPwm4Regs.CMPA.half.CMPA = (EPwm4Regs.TBPRD + 1) * u_prev / 3.3f;
131     LED_showfloat(2, out);
132     LED_showfloat(6, in);
133     AdcRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //清中断
134     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
135 }

```